

Condizioni in Python

Condizioni in Python

Come insegnare al programma a prendere decisioni con if, elif ed else.

Perché le condizioni?

Un programma che fa sempre le stesse cose, senza mai adattarsi alla situazione, è poco utile. Le **condizioni** permettono al programma di prendere decisioni: fare una cosa se qualcosa è vero, un'altra cosa se è falso.

È come un semaforo: se la luce è verde, vai; se è rossa, fermati.

Il blocco **if** — "se"

La parola **if** significa "se" in inglese. Dice a Python: "esegui questo blocco di codice solo se la condizione è vera".

```
if condizione:
    # questo codice viene eseguito solo se la condizione è vera
```

Esempio pratico:

```
eta = 18

if eta >= 18:
    print("Sei maggiorenne.")
```

Se **eta** vale 18 o più, viene stampato il messaggio. Se vale meno di 18, non succede nulla.

:::tip[Ricorda l'indentazione!] Il codice dentro l' **if** deve essere rientrato di 4 spazi. Questa è la regola fondamentale di Python. :::

Il blocco `else` — "altrimenti"

`else` significa "altrimenti". Si usa per definire cosa fare quando la condizione dell' `if` è falsa:

```
eta = 15

if eta >= 18:
    print("Sei maggiorenne.")
else:
    print("Sei minorenne.")
```

Python controlla la condizione. Se è vera, esegue il blocco `if`. Se è falsa, esegue il blocco `else`. Non possono essere eseguiti entrambi.

Il blocco `elif` — "altrimenti se"

`elif` è l'abbreviazione di "else if" (altrimenti se). Serve quando hai più di due possibilità da gestire:

```
voto = 75

if voto >= 90:
    print("Ottimo!")
elif voto >= 70:
    print("Buono!")
elif voto >= 60:
    print("Sufficiente.")
else:
    print("Insufficiente.")
```

Python controlla le condizioni dall'alto verso il basso e si ferma alla **prima** che è vera. Le altre vengono saltate.

Con `voto = 75` :

1. `voto >= 90` ? No (75 non è >= 90)
2. `voto >= 70` ? Sì! → stampa "Buono!" e si ferma

Condizioni annidate

Puoi mettere un `if` dentro un altro `if`. Si chiama annidamento:

```
eta = 20
ha_patente = True

if eta >= 18:
    if ha_patente:
        print("Puoi guidare.")
    else:
        print("Sei maggiorenne ma non hai la patente.")
else:
    print("Sei minorenne, non puoi guidare.")
```

Ogni livello di annidamento aggiunge altri 4 spazi di indentazione. Tieni d'occhio gli spazi!

Combinare condizioni con `and` e `or`

Invece di annidare, spesso puoi combinare due condizioni in una sola riga:

```
eta = 20
ha_patente = True

# Con 'and': entrambe devono essere vere
if eta >= 18 and ha_patente:
    print("Puoi guidare.")

# Con 'or': basta che una sia vera
eta = 16
con_accompagnatore = True

if eta >= 18 or con_accompagnatore:
    print("Puoi entrare.")
```

Controllare se un valore è in una lista

L'operatore `in` è molto comodo per verificare se un elemento è presente in una sequenza:

```
frutta_disponibile = ["mela", "banana", "uva"]
scelta = "mela"

if scelta in frutta_disponibile:
    print("Ottima scelta, ce l'abbiamo!")
else:
    print("Mi dispiace, quella frutta non c'è.")
```

Esempio completo

```
numero = int(input("Inserisci un numero: "))

if numero > 0:
    print("Il numero è positivo.")
elif numero < 0:
    print("Il numero è negativo.")
else:
    print("Il numero è zero.")
```

Con questo programma:

- Se scrivi `5` → "Il numero è positivo."
- Se scrivi `-3` → "Il numero è negativo."
- Se scrivi `0` → "Il numero è zero."